

Databricks Unity Catalog & Governance Guide

Namespaces, access control, masking, lineage and Delta Sharing

Unity Catalog centralises identity, permissions, lineage and auditing across every workspace in a Databricks account. This guide covers the three-level namespace, the privilege model, managed vs external tables and storage credentials, dynamic column masking and row-level filtering, automated lineage, audit logging, and sharing data across clouds with Delta Sharing.

Vivek Pandey — Data Engineer
Azure · Databricks · PySpark · Delta Lake
vp171097.github.io/DevWebsite

Contents

1. What Unity Catalog replaces
2. The three-level namespace
3. Metastore, storage credentials and external locations
4. Managed vs external tables
5. Identity: users, groups and service principals
6. The privilege model
7. Row-level security and column masking
8. Lineage and discovery
9. Audit logging and monitoring
10. Delta Sharing
11. Migration notes and common pitfalls

1. What Unity Catalog replaces

Before Unity Catalog, governance in Databricks was per-workspace: each workspace had its own Hive metastore, table ACLs were bound to a cluster's access mode, and credentials were mount points or cluster-scoped secrets. Two workspaces reading the same data had two independent permission models, and nothing traced which notebook read which column.

Concern	Legacy Hive metastore	Unity Catalog
Scope	One metastore per workspace	One metastore per region, shared by workspaces
Namespace	schema.table	catalog.schema.table
Identity	Workspace-local users/groups	Account-level principals
Storage access	Mounts and cluster secrets	Storage credentials + external locations
Lineage	None	Automatic, to column level
Audit	Partial, per workspace	Account-level audit logs
Sharing	Copy the data	Delta Sharing, no copy

2. The three-level namespace

Every securable is addressed as **catalog.schema.object**. The extra level is what allows one metastore to serve many teams and environments without name collisions.

```

portfolio                                <- catalog (environment or domain)
  ■■■ ecom_bronze                        <- schema
  ■   ■■■ orders                        <- table
  ■   ■■■ order_items
  ■■■ ecom_silver
  ■   ■■■ orders
  ■   ■■■ orders_quarantine
  ■■■ ecom_gold
    ■■■ dim_customer
    ■■■ fact_orders
    ■■■ agg_daily_sales

-- Fully qualified reference
SELECT * FROM portfolio.ecom_gold.agg_daily_sales;

-- Or set the context
USE CATALOG portfolio;
USE SCHEMA ecom_gold;

```

Catalog strategy The two workable patterns are one catalog per environment (dev, staging, prod) or one per business domain (sales, finance). Environment-based is simpler to reason about for promotion; domain-based suits a data-mesh ownership model. Mixing both inconsistently is what creates confusion later.

3. Metastore, storage credentials and external locations

Unity Catalog separates *who may access storage* from *where that storage is*. A storage credential wraps a cloud identity; an external location binds that credential to a path. Users are then granted rights on the location, never on the raw credential.

```
-- 1. Credential: a managed identity / service principal, created once by an admin
CREATE STORAGE CREDENTIAL cred_adls_portfolio
  WITH AZURE_MANAGED_IDENTITY (ACCESS_CONNECTOR_ID = '/subscriptions/.../accessConnectors/ac-dbx'
);

-- 2. External location: credential + path
CREATE EXTERNAL LOCATION ext_portfolio_lake
  URL 'abfss://lake@stdbxportfolio.dfs.core.windows.net/'
  WITH (STORAGE CREDENTIAL cred_adls_portfolio);

-- 3. Grant on the location, not the credential
GRANT READ FILES, WRITE FILES ON EXTERNAL LOCATION ext_portfolio_lake
  TO `data-engineers`;
```

Why this matters Nobody handles a key. Access is granted on paths, revocable centrally, and every use is attributable to a principal in the audit log — none of which is true of a mount point with an embedded secret.

4. Managed vs external tables

	Managed table	External table
Data location	Metastore-managed storage root	A path you specify
DROP TABLE	Deletes data as well	Removes metadata only; files remain
Maintenance	Predictive optimization can run automatically	You schedule OPTIMIZE/VACUUM
Best for	New pipelines built inside Databricks	Existing lakes, data shared with other engines

```
-- Managed: no LOCATION clause
CREATE TABLE portfolio.ecom_silver.orders (
  order_id STRING, customer_id STRING, purchased_at TIMESTAMP, price DECIMAL(12,2)
) USING DELTA;

-- External: explicit path inside a governed external location
CREATE TABLE portfolio.ecom_bronze.raw_events
  USING DELTA
  LOCATION 'abfss://lake@stdbxportfolio.dfs.core.windows.net/bronze/events';
```

Volumes Non-tabular files — CSV drops, PDFs, model artefacts — belong in a Volume, which is governed by the same grants: **CREATE VOLUME portfolio.ecom_landing.raw**; then read from **/Volumes/portfolio/ecom_landing/raw/**.

5. Identity: users, groups and service principals

Principals are defined once at account level, usually synced from Entra ID or Okta via SCIM, and are then visible in every workspace attached to the metastore.

```
-- Grant to groups, never to individuals: joiners and leavers are then an
-- identity-provider concern rather than a SQL migration.
GRANT USE CATALOG ON CATALOG portfolio TO `data-analysts`;
GRANT USE SCHEMA  ON SCHEMA portfolio.ecom_gold TO `data-analysts`;
GRANT SELECT      ON SCHEMA portfolio.ecom_gold TO `data-analysts`;

-- Jobs run as a service principal, not as the engineer who wrote them
GRANT MODIFY ON SCHEMA portfolio.ecom_silver TO `sp-ecom-pipeline`;
```

6. The privilege model

Privileges inherit down the hierarchy: a grant on a catalog applies to its schemas and their tables. Access also requires traversal — **USE CATALOG** and **USE SCHEMA** — which is the most common cause of an unexpected permission error.

Privilege	Applies to	Allows
USE CATALOG	Catalog	Traverse into the catalog (not read data)
USE SCHEMA	Schema	Traverse into the schema
SELECT	Table, view, volume	Read data
MODIFY	Table	INSERT, UPDATE, DELETE, MERGE
CREATE TABLE	Schema	Create tables in that schema
ALL PRIVILEGES	Any securable	Every privilege, including future ones
MANAGE	Any securable	Change ownership and grants

```
-- A complete read grant needs all three levels
GRANT USE CATALOG ON CATALOG portfolio TO `bi-readers`;
GRANT USE SCHEMA  ON SCHEMA  portfolio.ecom_gold TO `bi-readers`;
GRANT SELECT      ON TABLE   portfolio.ecom_gold.agg_daily_sales TO `bi-readers`;

-- Review and revoke
SHOW GRANTS ON TABLE portfolio.ecom_gold.agg_daily_sales;
REVOKE SELECT ON TABLE portfolio.ecom_gold.agg_daily_sales FROM `bi-readers`;
```

Ownership The owner of an object has full control regardless of grants. Set ownership to a group so access does not disappear when an individual leaves: **ALTER TABLE ... OWNER TO `data-platform`**;

7. Row-level security and column masking

Rather than maintaining one filtered view per audience, attach a function to the table so every reader sees the subset and the columns they are entitled to.

Dynamic column masking

```
CREATE OR REPLACE FUNCTION portfolio.security.mask_email(email STRING)
RETURN CASE
  WHEN is_account_group_member('pii-readers') THEN email
  ELSE regexp_replace(email, '^[^@]+' , '****')
END;

ALTER TABLE portfolio.ecom_silver.customers
  ALTER COLUMN email SET MASK portfolio.security.mask_email;
```

Row filters

```
CREATE OR REPLACE FUNCTION portfolio.security.region_filter(region STRING)
RETURN is_account_group_member('global-analysts')
  OR region = current_user_region();      -- e.g. a lookup on current_user()

ALTER TABLE portfolio.ecom_gold.fact_orders
  SET ROW FILTER portfolio.security.region_filter ON (region);
```

Function	Returns
current_user()	Email of the querying principal
is_account_group_member('g')	TRUE if the caller is in account group g
is_member('g')	Workspace-level group check (legacy)

Test with a real account Masks and filters do not apply to the object owner in some configurations, so verifying as yourself can give a false pass. Validate with an account that holds only the target group's privileges.

Tagging sensitive data

```
ALTER TABLE portfolio.ecom_silver.customers
  ALTER COLUMN email SET TAGS ('pii' = 'true', 'classification' = 'confidential');

-- Find everything classified as PII across the metastore
SELECT table_catalog, table_schema, table_name, column_name
FROM system.information_schema.column_tags
WHERE tag_name = 'pii';
```

8. Lineage and discovery

Unity Catalog captures lineage automatically for every query it runs — to column level — with no instrumentation. It answers two questions that otherwise take hours: what feeds this table, and what breaks if I change this column.

```
-- Upstream/downstream tables
SELECT source_table_full_name, target_table_full_name, event_time
FROM system.access.table_lineage
WHERE target_table_full_name = 'portfolio.ecom_gold.agg_daily_sales'
ORDER BY event_time DESC;

-- Column-level: who consumes customers.email?
SELECT source_column_name, target_table_full_name, target_column_name
FROM system.access.column_lineage
WHERE source_table_full_name = 'portfolio.ecom_silver.customers'
  AND source_column_name = 'email';
```

Discovery improves sharply when tables carry comments; they surface in Catalog Explorer and in search.

```
COMMENT ON TABLE portfolio.ecom_gold.agg_daily_sales IS
  'Daily delivered revenue, order counts and late-delivery rate. Refreshed hourly.';

ALTER TABLE portfolio.ecom_gold.agg_daily_sales
  ALTER COLUMN revenue COMMENT 'Sum of item price in BRL, delivered orders only';
```

9. Audit logging and monitoring

System tables expose account activity as queryable Delta tables — who read what, which queries cost the most, and how permissions changed over time.

System table	Contains
system.access.audit	Every action: logins, queries, grant changes
system.access.table_lineage	Table-level lineage events
system.access.column_lineage	Column-level lineage events
system.billing.usage	DBU consumption per workspace/warehouse
system.query.history	Executed queries with duration and bytes scanned

```
-- Who read a PII-bearing table in the last 7 days?
SELECT user_identity.email, action_name, event_time, request_params.full_name_arg
FROM system.access.audit
WHERE service_name = 'unityCatalog'
  AND request_params.full_name_arg = 'portfolio.ecom_silver.customers'
  AND event_date >= current_date() - INTERVAL 7 DAYS
ORDER BY event_time DESC;

-- Permission changes – the first thing to check after an access incident
SELECT event_time, user_identity.email, action_name, request_params
FROM system.access.audit
WHERE action_name IN ('updatePermissions', 'createCatalog', 'deleteSchema')
  AND event_date >= current_date() - INTERVAL 30 DAYS;
```

10. Delta Sharing

Delta Sharing gives a recipient live read access to a table without copying it. The recipient may be on another cloud, or not on Databricks at all — an open protocol client can read the share from pandas or Spark.

```
-- Provider side
CREATE SHARE sales_share;
ALTER SHARE sales_share ADD TABLE portfolio.ecom_gold.agg_daily_sales;

-- Databricks-to-Databricks: identify the recipient by its sharing identifier
CREATE RECIPIENT partner_acme
  USING ID 'aws:us-west-2:19a84bee-....';

-- Open sharing: issue a credential file instead
CREATE RECIPIENT partner_open;
GRANT SELECT ON SHARE sales_share TO RECIPIENT partner_acme;
```

```
# Recipient side, no Databricks required
import delta_sharing
url = "config.share#sales_share.ecom_gold.agg_daily_sales"
df = delta_sharing.load_as_pandas(url)
```

Share views, not raw tables Expose a view that selects only the columns and rows a partner should see. Sharing a base table means every future column added to it is shared too.

11. Migration notes and common pitfalls

1. **Traversal privileges.** SELECT alone is not enough; without USE CATALOG and USE SCHEMA the table appears not to exist.
2. **Cluster access mode.** Unity Catalog requires a shared or single-user access-mode cluster. No-isolation clusters cannot see UC tables at all.
3. **Mounts do not carry over.** `/mnt/...` paths bypass UC governance. Replace them with external locations and Volumes rather than keeping both.
4. **hive_metastore remains visible.** Legacy tables live in the **hive_metastore** catalog; upgrade them explicitly rather than assuming migration happened.
5. **Grant to groups.** Individual grants accumulate into an unauditable mess and outlive the people they were made for.
6. **Group ownership.** Objects owned by a departing employee become orphaned; set ownership to a team group.
7. **Managed DROP deletes data.** On a managed table, DROP TABLE removes the files too — only time travel within the retention window can recover it.

```
-- Upgrading a legacy table in place
CREATE TABLE portfolio.ecom_bronze.orders
LIKE hive_metastore.default.orders COPY LOCATION;

-- Or clone it (copies data, leaves the original untouched)
CREATE TABLE portfolio.ecom_bronze.orders
DEEP CLONE hive_metastore.default.orders;
```

Sequence that works Create the metastore and storage credentials first, then catalogs and schemas, then sync groups from the identity provider, then grant at schema level, and only then migrate tables. Granting before the group structure exists produces permissions nobody can explain six months later.